Indira Gandhi Delhi Technical University for Women

Department of Information Technology

Summer Internship Program 2026

sansoftech

CAPSTONE PROJECT
DOCUMENTATION & SUBMISSION REPORT

Project Title	PDF Analyser
Team Name	Discoders
Project Track	AI / Generative AI
Team Members	Keerthana | Khanak Aggarwal | Rajrita Banik | Kashish Gupta | Manasvi Agarwal
Submission Date	22 July 2026
GitHub Repository	https://github.com/khanakaggarwal1204/Pdf_analyser
Demo / App Link	Local only — see Section 11 for setup instructions

This document is submitted in partial fulfilment of the capstone evaluation for SIP 2026.

1. Team & Project Information
2. Executive Summary
3. Problem Statement & Motivation
4. Literature Review & Background
5. Proposed Solution & Objectives
6. System Architecture
7. Technical Implementation
8. Dataset & Data Pipeline
9. Model / Pipeline Evaluation
10. Responsible AI & Ethics
11. Deployment & Usage Guide
12. Challenges & Limitations
13. Future Work
14. Conclusion References Appendices

1. Team & Project Information

Complete all fields accurately. This section is used for evaluation record-keeping and certificate issuance.

Project Title	PDF Analyser
Team Name & Number	Discoders | Team No. [88]
Project Track	AI / Generative AI
Team Leader	Keerthana | Roll No: 08901172025 | Dept: CSE-AI| Email: keerthana089btcseai25@igdtuw.ac.in
Member 2	Khanak Aggarwal | Roll No: 05801032025 | Dept: IT
Member 3	Rajrita Banik | Roll No: 15101172025 | Dept: CSE-AI
Member 4	Kashish Gupta | Roll No: 08601172025 | Dept: CSE-AI
Member 5	Manasvi Agarwal | Roll No: 0670103025 | Dept: IT

Submission Date	22 July 2026
GitHub Repository	https://github.com/khanakaggarwal1204/Pdf_analyser
Live Demo / App	Local only — see Section 11 for setup instructions
Primary LLM Used	To be confirmed. The repository's "About" description references an OpenAI model, but requirements.txt and the current codebase do not yet pin a specific LLM integration — please verify and record the exact model actually called (e.g. GPT-4o, GPT-4o-mini) before final submission.
Primary Framework	LangChain (per repository description). To be confirmed against the actual backend code once the LLM integration is finalised.

2. Executive Summary

Cover: (a) the problem addressed, (b) the AI solution built, (c) key technology choices, (d) main results achieved, (e) target users and impact.

Reading and understanding lengthy PDF documents — research papers, reports, legal documents, manuals, and study material — is time-consuming, and users often spend significant time searching for specific information buried in long files. PDF Analyser addresses this by letting users upload a PDF and ask natural-language questions about its contents. Rather than relying solely on the language model's general knowledge, the system is designed around a Retrieval-Augmented Generation (RAG) pipeline: uploaded documents are parsed, cleaned, split into chunks, and converted into vector embeddings, which are stored for semantic search. When a user asks a question, the system retrieves the most relevant chunks from the document and passes them to the LLM as grounding context before generating an answer, improving factual accuracy and reducing hallucination compared to an ungrounded chatbot.

The project is built as a FastAPI backend paired with a lightweight HTML/CSS/JavaScript frontend, with LangChain, ChromaDB, and an OpenAI model intended as the core AI stack (per the project's repository description). At the current stage, the frontend is functional and the backend scaffolding (FastAPI, SQLAlchemy, file handling) is in place, while the live RAG pipeline (chunking, embeddings, retrieval, generation) is still being wired up end-to-end — the frontend currently runs against mocked responses while backend integration is completed.

The target users are students, researchers, and professionals who work with large PDF documents and need to extract specific information quickly instead of reading pages manually. Once fully integrated, the expected impact is a meaningful reduction in the time spent manually searching documents, with answers grounded in the user's own uploaded material rather than the model's general knowledge.

3. Problem Statement & Motivation

3.1 Problem Statement

Reading and understanding lengthy PDF documents such as research papers, reports, legal documents, manuals, and study material is time-consuming. Users — including students, researchers, and professionals — often spend significant time searching for specific information scattered across long documents, with no quick way to ask a direct question and get a grounded answer from the document itself.

3.2 Motivation & Real-World Relevance

Instead of manually reading hundreds of pages to find one relevant fact, users can instantly obtain relevant information through intelligent document understanding, improving productivity and saving time. For example, a student revising from a 100-page lecture PDF, or a professional searching a lengthy contract for a

specific clause, can simply ask the system directly instead of scanning the whole document — this is the core use case PDF Analyser targets.

3.3 Scope & Constraints

- In scope: Upload PDF documents
- In scope: Extract document text
- In scope: Analyze document content
- In scope: Answer user queries based on uploaded PDFs
- In scope: Generate context-aware responses
- Out of scope: Internet search
- Out of scope: Editing PDFs
- Out of scope: Knowledge beyond the uploaded documents
- Out of scope: Complex image understanding inside PDFs (unless implemented)

4. Literature Review & Background

4.1 Related Work & Existing Solutions

[1] Vaswani et al. (2017), “Attention Is All You Need” — introduces the Transformer architecture underlying modern LLMs; foundational but not a document-QA system in itself.
[2] Lewis et al. (2020), “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” — proposes the RAG paradigm PDF Analyser is built on; general-purpose and not tuned for PDF-specific parsing challenges (tables, scans, layout).
[3] Karpukhin et al. (2020), “Dense Passage Retrieval for Open-Domain Question Answering” — dense retrieval approach informing the retrieval-layer design; typically requires a large labelled corpus to train a strong retriever from scratch.
[4] Devlin et al. (2019), “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding” — underlies many embedding models used for semantic search; on its own it is an encoder, not a full QA pipeline.
[5] Johnson et al. (2017), “FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors” — a widely used vector-search library; an alternative to the ChromaDB vector store used in this project.

4.2 Key Technologies & Concepts

- Retrieval-Augmented Generation (RAG): combines a retriever and a generator so the LLM’s answer is grounded in retrieved document chunks rather than parametric memory alone, directly reducing hallucination.
- Vector embeddings & semantic search: text chunks are converted into dense vectors so that retrieval can match a question to relevant passages by meaning rather than exact keywords.
- Chunking: long documents must be split into smaller, overlapping passages so that each retrieved unit fits within the LLM’s context window while preserving enough surrounding meaning.

4.3 Gap Addressed by This Project

Generic chatbots answer from the LLM’s pre-trained knowledge and cannot reliably discuss the specific contents of a document the user just uploaded. PDF Analyser closes this gap by grounding every answer in the user’s own uploaded PDF through retrieval, so responses reflect the actual document rather than the model’s general (and potentially outdated or irrelevant) knowledge.

5. Proposed Solution & Objectives

5.1 Solution Overview

PDF Analyser is an AI-powered document assistant that lets users upload PDF files and ask natural-language questions about their contents. Rather than relying solely on the language model's general knowledge, the system retrieves relevant sections from the uploaded document and generates grounded, context-aware responses. The user flow is simple: upload a PDF, wait for it to be processed (text extraction, chunking, embedding), and then chat with the document directly. The main value proposition is faster, more trustworthy access to information already contained in a document the user has in hand, without needing to read it end-to-end.

5.2 Key Objectives

1. Allow users to upload a PDF and reliably extract its text content.
2. Chunk and embed extracted text so relevant passages can be retrieved for a given query.
3. Answer natural-language user queries using only retrieved, document-grounded context.
4. Keep answers scoped to the uploaded document (no unrelated internet search or outside knowledge).
5. Provide a usable, responsive chat interface for uploading documents and asking questions.

5.3 Innovation & Differentiation

Rather than simply forwarding a user's question to an LLM, PDF Analyser adds a retrieval layer in between: the document is parsed, chunked, embedded, and indexed, so that only the most relevant passages are passed to the LLM as context. This grounding step is what differentiates the system from a plain LLM API call — it constrains the model's answer to the actual document content and makes responses easier to trust and verify.

6. System Architecture

6.1 Architecture Diagram

[Insert Architecture Diagram Here — export from draw.io, Lucidchart, Excalidraw, or any diagramming tool as PNG/JPG]
The flow to diagram is: User Interface → PDF Processing Layer → Text Preprocessing → Embedding Generation → Vector Storage → Retrieval Layer → LLM Layer → Response back to the User Interface.
Figure 1: System Architecture Diagram

6.2 Component Description

Component	Description	Technology Used
User Interface	Allows users to upload PDF files and ask questions.	HTML / CSS / JavaScript frontend
PDF Processing Layer	Extracts text from uploaded PDF documents.	[PDF parser to be confirmed — e.g. PyMuPDF / pypdf / pdfplumber]
Text Preprocessing	Cleans and divides extracted text into smaller chunks.	[To be confirmed]
Embedding Generation	Converts text chunks into vector embeddings.	[Embedding model to be confirmed]
Vector Storage	Stores embeddings for efficient semantic search.	ChromaDB (per repository description)
Retrieval Layer	Retrieves the most relevant document chunks based on the user's query.	LangChain (per repository description)
LLM Layer	Uses retrieved context to generate accurate responses.	[LLM to be confirmed — repository description references OpenAI]

7. Technical Implementation

7.1 Technology Stack

LLM / AI Model	To be confirmed — repository description references an OpenAI model; verify exact model name used
Frameworks	LangChain (per repository description)
Embedding Model	To be confirmed from document_processor/embeddings.py
Vector Database	ChromaDB (per repository description)
Fine-Tuning Method	N/A
Frontend / UI	HTML / CSS / JavaScript (no build step)
Serving / Deployment	FastAPI backend, run locally
Other Libraries	FastAPI, Uvicorn, SQLAlchemy, python-multipart, pydantic, python-dotenv, requests, aiofiles
Dev Environment	Python 3.x (exact version to be confirmed); VS Code / PyCharm (to be confirmed); no GPU required for the current implementation

7.2 Implementation Details

7.2.1 PDF Processing & Text Extraction

Uploaded PDFs are parsed to extract their raw text content via the PDF processing layer (document_processor / read_pdf.py in the repository). The exact parser library (e.g. PyMuPDF, pypdf, or pdfplumber) should be confirmed from the code before final submission, as it affects the "Known Limitations" noted in Section 8.3 (scanned/image-heavy PDFs).

7.2.2 Chunking & Embedding

Extracted text is cleaned and split into smaller chunks, which are then converted into vector embeddings (see document_processor/embeddings.py) and stored for semantic search. The specific chunk size/overlap and embedding model should be recorded here once confirmed from the code.

7.2.3 Retrieval & Generation Layer

At query time, the system retrieves the most relevant chunks for the user's question from the vector store and passes them as context to the LLM, which generates a grounded response. This retrieval-then-generate pattern is what keeps answers scoped to the uploaded document rather than the model's general knowledge.

7.2.4 Backend & Frontend

The backend is implemented with FastAPI (with SQLAlchemy for persistence), exposing endpoints for document upload and question answering. The frontend is a dependency-free HTML/CSS/JS app (index.html, style.css, app.js) that calls these endpoints. As of this submission, the frontend's API calls are mocked pending full backend integration — this should be completed and re-verified before final submission.

8. Dataset & Data Pipeline

8.1 Dataset Description

Dataset Name / Source	User-uploaded PDF documents (no fixed dataset — data is supplied at runtime by each user)
Dataset Type	Unstructured textual data extracted from PDF files
Size	Variable — depends on the PDF(s) a user uploads
Domain	General-purpose documents, including academic, technical, legal, and business PDFs
Licence / Rights	N/A — documents are supplied by the end user, not distributed by the project

Train / Val / Test Split	N/A — this is a retrieval system operating on user-supplied documents, not a trained/fine-tuned model

8.2 Data Preprocessing

1. Text extraction from the uploaded PDF.
2. Cleaning of the extracted text.
3. Chunking of the cleaned text into smaller passages.
4. Embedding generation for each chunk and storage in the vector store.

8.3 Data Quality & Limitations

Poor-quality scans reduce extraction accuracy, and image-heavy PDFs may not be fully understood by the current pipeline since there is no dedicated OCR or image-understanding step. Overall answer quality is therefore dependent on how “text-native” the uploaded PDF is; scanned documents without OCR will yield incomplete or empty extracted text.

9. Model / Pipeline Evaluation

9.1 Evaluation Metrics

Metric	Definition / Tool	Why Chosen
Answer Accuracy	Whether the generated answer correctly reflects the uploaded document’s content (to be measured via manual/spot review)	Directly measures whether the core promise of the tool — grounded answers — is being met
Retrieval Relevance	Whether the chunks retrieved for a query are actually relevant to it (to be measured)	Retrieval quality upper-bounds answer quality in a RAG system
Response Time	End-to-end time from question submission to displayed answer (to be measured)	Usability depends on the system feeling responsive
User Satisfaction	Qualitative feedback from test users on answer usefulness (to be measured)	Captures perceived value beyond raw accuracy numbers

9.2 Evaluation Results

Keep as placeholders unless actual testing has been performed.

Metric	Baseline / Target	Your System	Achieved?
Answer Accuracy	To be measured	To be measured	☐ Yes / ☐ No
Retrieval Relevance	To be measured	To be measured	☐ Yes / ☐ No
Response Time	To be measured	To be measured	☐ Yes / ☐ No
User Satisfaction	To be measured	To be measured	☐ Yes / ☐ No

9.3 Failure Analysis

Failure Case 1: Poor-quality scanned PDFs produce incomplete answers, since the text-extraction step does not currently include OCR and cannot recover text from image-only pages.
Failure Case 2: Diagram- or image-based questions cannot always be answered correctly, and questions requiring knowledge beyond the uploaded PDF may produce incomplete responses, since the system is intentionally scoped to the document’s own text content and does not perform internet search.

10. Responsible AI & Ethics

10.1 Bias & Fairness

Responses are generated only from the uploaded document, which reduces (but does not eliminate) dependence on the underlying pre-trained model's own biases, since the grounding context comes from the user's own material. No formal bias testing (e.g. ToxiGen-style prompts or demographic parity checks) has been run yet; this should be added before final submission if time permits.

10.2 Hallucination Risk & Mitigation

Relevant document sections are retrieved before generating answers, which improves factual grounding compared to an ungrounded chatbot. No automated faithfulness scoring (e.g. RAGAS) has been implemented yet; this is listed under Future Work.

10.3 Data Privacy & PII

Uploaded documents remain user-specific and are not used for model training. No dedicated PII-detection step (e.g. Presidio) is currently implemented; this should be considered before handling sensitive documents in a production setting.

10.4 Dataset & Content Licensing

Only properly licensed libraries, APIs, and software components are used in the project. Uploaded documents themselves are supplied by the end user and are not redistributed by the system.

10.5 Human-in-the-Loop

Users verify responses themselves and can inspect the retrieved document content that supports each answer, providing a lightweight human-in-the-loop check on the system's output.

11. Deployment & Usage Guide

11.1 System Requirements

Python Version	Python 3.x (exact version to be confirmed from the development environment / requirements.txt)
OS / Platform	Not specified — assumed cross-platform (Windows / macOS / Linux)
GPU Requirement	Not required for the current implementation
API Keys Required	Depends on the LLM provider used once integration is finalised (e.g. OPENAI_API_KEY)
Disk Space	Minimal — depends on size of uploaded PDFs and the local vector store

11.2 Installation & Setup

1. Clone the GitHub repository: git clone https://github.com/khanakaggarwal1204/Pdf_analyser
2. Install project dependencies: pip install -r requirements.txt
3. Configure any required API keys (e.g. copy .env.example to .env if present and fill in values).
4. Run the application (e.g. python main.py, or serve the frontend with python -m http.server 5500).
5. Upload a PDF through the interface.
6. Ask questions about the document in natural language.

11.3 Usage Instructions

1. Open the application in a browser.
2. Upload a PDF document.
3. Wait for the document to be processed (text extraction, chunking, embedding).
4. Type a question about the document into the chat input.

5. The system retrieves relevant chunks and generates a context-aware response.
6. Review the answer and, where available, the supporting retrieved content.

12. Challenges & Limitations

12.1 Technical Challenges Faced

- Accurate text extraction from PDFs, particularly scanned or poorly formatted files.
- Maintaining useful context for long documents within the LLM's context window.
- Reducing hallucination in generated answers.
- Building an efficient semantic retrieval pipeline.

12.2 Current Limitations

- Limited support for scanned documents (no OCR step currently implemented).
- No advanced image or diagram understanding inside PDFs.
- Answer accuracy depends heavily on the quality of the source document.
- The frontend currently runs against mocked responses pending full backend/LLM integration.

13. Future Work

- Multi-document querying (asking questions across several uploaded PDFs at once).
- OCR integration for scanned documents.
- Citation generation, so answers point back to the specific page/section they came from.
- Better table understanding within PDFs.
- Voice-based interaction.
- Conversation memory across multiple turns.
- An improved evaluation dashboard (e.g. tracking accuracy, retrieval relevance, and response time over time).

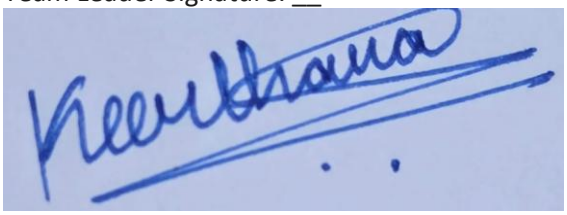
14. Conclusion

PDF Analyser sets out to make long PDF documents searchable through natural-language questions instead of manual reading, grounding its answers in retrieved document content rather than the LLM's general knowledge alone. The core pipeline — upload, text extraction, chunking, embedding, retrieval, and generation — is designed and scaffolded, with a working frontend and a FastAPI-based backend structure in place. At this stage, full end-to-end integration between the frontend and a live retrieval/LLM pipeline is still in progress, and quantitative evaluation (accuracy, retrieval relevance, response time, user satisfaction) has not yet been carried out and remains a placeholder for the next iteration. The team's key learning has been the practical difference between designing a RAG architecture and fully wiring it end-to-end, including handling document quality issues such as scans and image-heavy pages. Completing the live LLM/embedding integration and running the evaluation in Section 9 are the immediate next steps before this can be considered feature-complete.

Team Declaration

We hereby declare that this capstone project documentation represents our team's own original work. All external sources, datasets, models, and tools have been properly acknowledged. No part of this submission has been plagiarised from any other source.

Team Leader Signature: __



Name: Keerthana
Date: _____

Mentor Signature:
_____

Name: _____
Date: _____

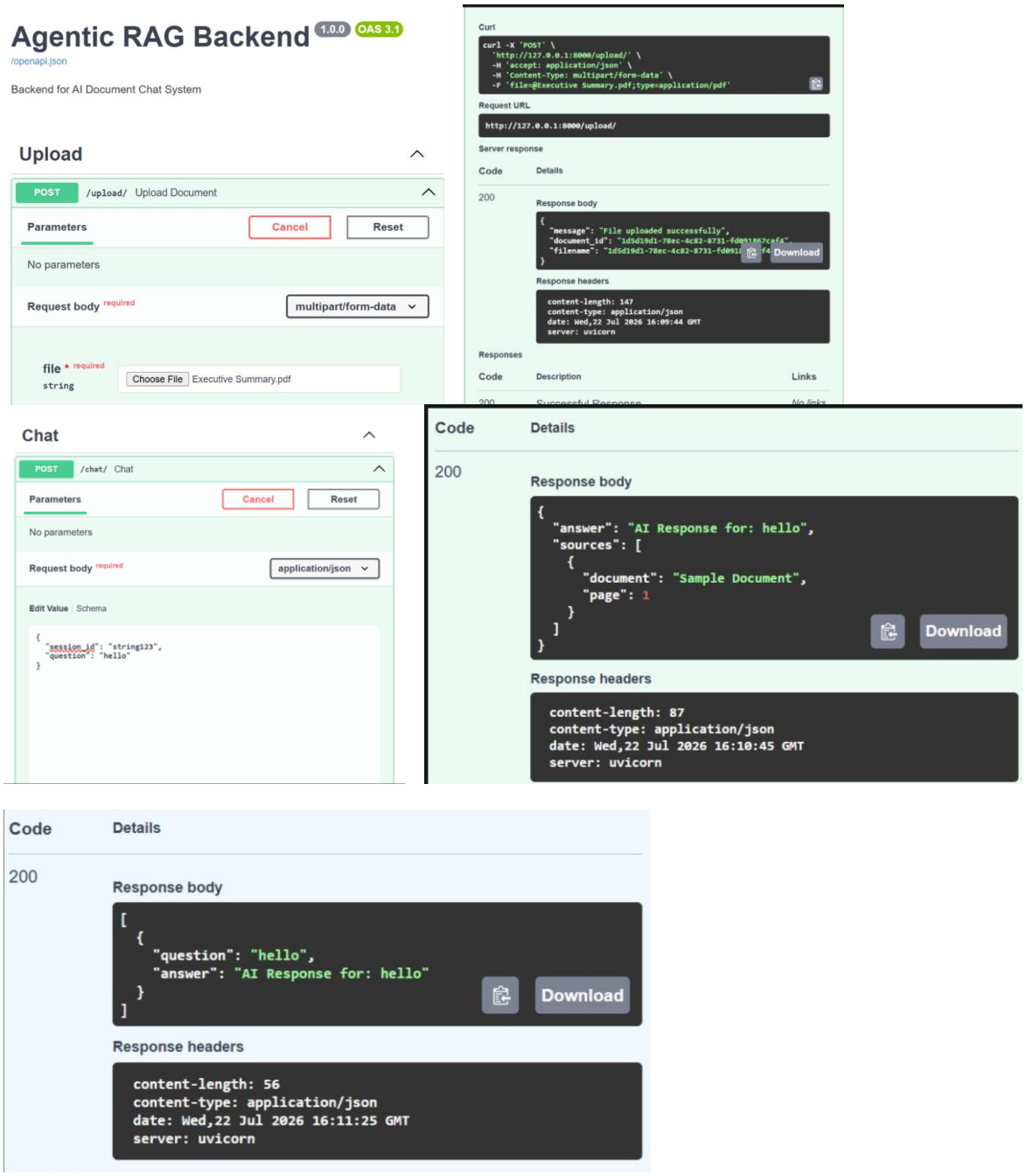
Use IEEE or APA citation format consistently throughout. Number each reference and cite them in the text as [1], [2], etc.

[1] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
[3] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," *Proceedings of EMNLP*, 2020.
[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proceedings of NAACL-HLT*, 2019.
[5] J. Johnson, M. Douze, and H. Jégou, "FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors," *IEEE Transactions on Big Data*, 2017.
[6] LangChain Documentation, https://python.langchain.com/
[7] ChromaDB Documentation, https://docs.trychroma.com/
[8] OpenAI API Documentation, https://platform.openai.com/docs

Appendix A — GitHub Repository Structure

```
Pdf_analyser/
 ├── app/
 ├── backend/
 ├── data/
 ├── document_processor/
 ├── uploads/
 ├── .gitignore
 ├── README.md
 ├── agentic_rag.db
 ├── app.js
 ├── index.html
 ├── main.py
 ├── requirements.txt
 └── style.css
```

Appendix B — Sample Outputs / Screenshots



Appendix C — Evaluation Notebook

[Link to evaluation notebook — to be added once evaluation in Section 9 is completed]

Appendix D — Team Contribution Statement

Member	Contribution
Keerthana Madhav (Team Leader)	Implemented the RAG pipeline, including document processing, vector database integration, semantic retrieval, and generator module integration.

Member	Contribution
Khanak Aggarwal	Responsible for project research, GitHub repository management, presentation development, and brainstorming research paper ideas while supporting overall project documentation and knowledge organization.
Rajrita Banik	Worked on the backend of CampusConnect AI using FastAPI, developing APIs for document processing, RAG integration, PDF upload workflows, embeddings, vector database storage, and frontend integration.
Kashish Singh	I developed the backend using FastAPI by creating APIs for document upload, listing, deletion, chat, and chat history. I integrated the database with SQLAlchemy and SQLite, handled document text extraction, tested the APIs, fixed bugs, and supported backend integration.
Manasvi Agarwal	I worked on the frontend development of the project by designing and implementing the user interface. I developed responsive UI components, integrated them with the backend APIs, tested the frontend for functionality and usability, fixed UI issues and bugs, and helped improve the overall user experience.

Tick every box before submitting. An incomplete submission will be flagged for review and may affect your evaluation score.

#	Deliverable / Requirement	✓	
D1	Cover page completed with all team and project details	☐	
D2	Executive summary written (200–300 words)	☑	
D3	Problem statement is specific and grounded in a real need	☑	
D4	Literature review cites at least 5 relevant references	☑	
D5	System architecture diagram is included and labelled	☐	
D6	Component description table is fully completed	✓	☐
D7	Technology stack table is fully completed	✓	☐
D8	All major implementation modules are described in Section 7	☐	
D9	Dataset description and preprocessing steps are documented	☑	
D10	At least 2 quantitative evaluation metrics are reported	☐	
D11	Baseline comparison is included in the results table	☐	
D12	At least 2 failure cases are analysed in Section 9.3	☑	
D13	Responsible AI section covers bias, hallucination, and PII	☑	
D14	Installation and setup instructions can be followed without assistance	☐	
D15	All references are cited in IEEE or APA format	☑	
D16	GitHub repository is public or shared with the mentor	☑	
D17	README.md in the repository matches the setup instructions in Section 11	☐	
D18	Evaluation notebook is included in the repository and is runnable	☐	
D19	Team contribution statement is completed in Appendix D	☑	
D20	Team declaration is signed by the team leader	☐	

Submit this completed document as a single .docx or PDF file named: Discoders_AI-GenerativeAI_CapstoneDoc_SIP2026.docx to [submission email / portal link].